PROJECT HANDOVER

Pou Hono

The registration website for NZ Tech Week 2027
SACTH (South Auckland Creative Tech Hub) & The Cause Collective

Version	1.2 (final)
Date	12 October 2026
Prepared by	Yahya El-Saddik
Codebase	GitHub repository `PouHonoSite : main` at `70fda57` or later
Status	Feature-complete, security-reviewed, all fixes merged; not yet on production hosting

Contents

Part A is written for SACTH and The Cause Collective and needs no technical knowledge. Where a step needs a developer, it says so. Part B is a reference for whoever maintains or hosts the website next.

PART A — FOR SACTH & THE CAUSE COLLECTIVE

1 Summary and where things stand
2 What has been delivered
3 How it works
4 Keeping the data safe
5 Going live on Railway, step by step
6 Setting up the confirmation email
7 The database: is it big enough?
8 Languages and translations
9 Personalising the website: what to edit where
10 Before going live: checklist
11 Known limitations and recommended next steps
12 Not included in this project
13 Contacts and ownership

PART B — TECHNICAL REFERENCE

14 Architecture, file map and dependencies
15 Database
16 Access control and API reference
17 Setup, configuration and operations
18 Making technical changes
19 Repository status

APPENDICES

A Screenshots
B Event-day check-in guide for staff

What changed since version 1.1 (5 October 2026)

All the fixes from the October security review are now part of the main version, so nothing is waiting to be merged. Since 1.1 the site has gained the five real workshops, year groups on the teacher form, a school-name search, a PDF export for staff, a confirmation email for teachers, and translations on the home page. This version also adds full step-by-step guides for going live on Railway (section 5), email (section 6), the database (section 7), languages (section 8) and personalisation (section 9), and corrects two statements in 1.1 that did not match the website (see section 11).

PART A For SACTH & The Cause Collective. No technical knowledge needed.

1 Summary and where things stand

Pou Hono is the registration website for NZ Tech Week 2027, run by SACTH and The Cause Collective for South Auckland schools, youth groups and individuals.

A **teacher** registers their class and is given two things: a link and QR code to share with their students, and a private link to their own class portal. They are also sent both by email once email is switched on. **Students** open the shared link and register themselves, and they are grouped under their teacher's class automatically. Teachers can also type students in themselves. **Individuals** who are not part of a school group can register on their own. **SACTH staff** see everything in a password-protected admin dashboard, can download it as a PDF, and use the dashboard to check classes in on the day.

Why it was built in-house

The website collects children's names, ages, year groups and allergy or health information. Rather than putting that into a third-party sign-up service, Pou Hono keeps all of it in a single database file that sits with the website itself. This is the **data sovereignty** reason the project exists, and it shaped most of the design decisions in this document. The site also loads a Google map. Its optional email provider receives the teacher's own name, email address, school name, event details, links and QR code (section 6). Console email previews also put teacher details in hosting logs; treat those logs as confidential. No child's details are emailed.

Where things stand

Done and in the main version

- Teacher, student and individual registration
- Shareable links and QR codes; private teacher portal
- The five real workshops
- Year groups on the teacher form, and school-name suggestions
- Admin dashboard: filters, check-in, editing and deleting classes, resetting teacher links, PDF export
- Confirmation email to teachers (built and tested; needs an email account to switch on)
- Four languages on the public pages (draft translations)
- Security review, with every fix merged
- Branding, logos and event photo gallery

Needed before launch

- Move to Railway on the **Pro** plan (section 5). **Vercel cannot keep registrations.**
- An email provider and its DNS records (section 6)
- Confirmed event dates and event-day details
- Fluent-speaker review of te reo Māori, Samoan and Tongan text
- Real admin password; demo data switched off
- Privacy statement and parent/caregiver consent (section 11)
- A load test and daily backups (section 7)

2 What has been delivered

Everything below is working in the main version. Screenshots are in Appendix A.

Who	What they can do
Anyone visiting `Home, FAQ`	• Home page: about the event and SACTH, a live count of registered classes, a $500 prize giveaway banner, a photo gallery from past events with full-size viewing, and a map to 15 Earl Richardson Avenue, Wiri. • The **five workshops**, as flip cards with a short description on the back: *AI Fundamentals Workshop* (hosted by Microsoft), *Graphic Design Workshop* (The Cause Collective), *Build and Battle Robots Workshop* (Robot Rampage), *How to Hack a Bank (Ethical Hacking Workshop)* (Whitecliffe) and *DJ'ing Basics Workshop* (Serato). • FAQ page with seven common questions. • The SACTH and The Cause Collective logos in the header; contact email, phone number and address in the footer.
Teachers `Register page`	• Three-step registration. Step 1: name, email, school, number of students and adults, **which year groups are coming** (at least one must be ticked), whether all students are attending (and if not, how many are not), and which of the three event days. Step 2: accept the terms. Step 3: confirmation. • The **school name box suggests South Auckland schools** as they type (74 schools in Manurewa, Ōtara-Papatoetoe, Māngere, Papakura and wider Manukau). Any other name can still be typed in full. • The confirmation screen gives a **student link and QR code** to share, and a separate **portal link** to bookmark. They can start adding students by hand straight away. • A **confirmation email** with both links, the QR code and a "save the date" box, once email is switched on (section 6).
Teacher class portal `Private link`	Shows the class details (including year groups and event day), the student link and QR code again, every student signed up so far (including allergies), a form to add a student by hand, and a button to remove a student.
Students `Shared link / QR`	A short form: name, age, year group, allergies or health conditions (optional), and a choice of one of the five workshops. Before submitting they tick a short statement about how their information is used. Works well on a phone. Students are not asked for an email address and get no email.
Individuals	Choose "I'm an Individual" on the Register page and fill in the same short form, without needing a school or teacher. Adults can register (ages up to 120, year group "18+").

SACTH staff (admin) `Admin dashboard`	• Password-protected sign-in that ends after 8 hours, or straight away on logout. • Headline figures: total registrations, total students expected, total adults, most popular workshop. • A table of every registered class, including its year groups, with filters by school name, workshop, year group, age range and arrival status. • **Check-in:** click a class's status to switch it between *Pending* and *Arrived*. • **Edit or delete a class.** Editing covers school, contact, email, numbers, event day and notes (not year groups). Deleting asks for confirmation first, and also removes that class's students. • **Reset a teacher's portal link** if it has been lost or shared by mistake. The old link stops working and the admin gets a new one to send to the teacher. • **Export PDF:** one A4 landscape file of every student (name, school, year level, workshop, when they registered) and every teacher registration (name, school, email, when they registered), with times in New Zealand time. It deliberately leaves out ages and allergies. It always contains everything, whatever filters are set. • A pie chart of students by workshop, which updates with the filters. Staff **cannot** edit or remove an individual student from the dashboard. Only the teacher can remove a student from their own class. See section 11.

3 How it works

The full journey, from a teacher signing up to a class arriving on the day.

1. Teacher registers
Class details, year groups and an event day.

→

2. Gets two links
On screen and by email: a student link and QR, and a private portal link.

→

3. Students sign up
Through the link or QR, or the teacher adds them. Each picks a workshop.

→

4. Admin sees it all
Numbers, filters, chart and PDF. Refresh to see new sign-ups.

→

5. Event day
Staff mark each class *Arrived*.

Two links, two different codes

When a teacher registers, the website creates **two** long random codes for the class, one for each link:

Link	Who it is for	What it can do
`…/join/<student code>`	Students. Share with the class or print as a QR code.	Adds a student to that class and shows its school name. It can't open the teacher portal or show student details.
`…/register/<teacher code>`	The teacher only. **Do not share.**	Opens the teacher's class portal: the class list, including allergies, and adding or removing students.

There are no teacher accounts or passwords. Having the teacher code *is* the permission, so a teacher can start straight away with nothing to set up. The codes are far too long to guess. If a portal link is lost or shared by mistake, an admin can reset it from the dashboard: the old link stops working immediately, and the student link and QR code keep working. (The reset link is shown to the admin to pass on; it is not emailed automatically.)

Individuals

Behind the scenes, everyone who registers as an individual is grouped under one shared record called "Individual / Walk-in". It never appears as a class in the admin table, and it can never open a teacher portal. Individual sign-ups are still counted in the dashboard figures and the chart, and appear in the PDF with "Individual" as their school.

What the admin figures mean

- **Total registrations:** the number of classes that have registered (individuals not included).
- **Total students:** the expected number of students: each class's booked number minus those not attending, plus individual sign-ups.
- **Most popular session** and the **pie chart** come from the students who have actually signed up, because a workshop is chosen by each student, not by the class.

4 Keeping the data safe

The website holds children's personal and health information, so access is deliberately tight.

Who can see what

Level	Can	Cannot
PUBLIC anyone	Register a class. Sign a student up, if they have a valid student link.	See student or teacher details. A student link shows the school name; the home page receives only a count.
TEACHER has a portal link	See and manage **their own class only**.	See or change any other class, or the individual sign-ups.
ADMIN has the password	See everything, manage classes, export the PDF.	Edit or remove a single student (section 11).

Protections already in place

- **The admin password is never stored.** Only a scrambled, one-way version of it is kept (a "bcrypt hash"), so the password can't be read back out of the server's settings.
- **Admin sign-in expires and can be revoked.** Each sign-in gets its own random session, recorded on the server, which ends after 8 hours, on logout, or when the admin password is changed. Logging out also clears the dashboard from the screen.
- **Limits on repeated attempts.** 10 admin login attempts per 15 minutes and 20 class registrations per hour from one internet address. Student sign-ups allow 100 an hour per class and internet address (60 for individuals), with a ceiling of 600 per address. Teachers can add 200 an hour per class. These slow automated password guessing and junk sign-ups.
- **Codes are never handed out.** A class's codes appear inside its links: on the confirmation screen and in the teacher's email, in the student link returned to the teacher portal, and in a new portal link returned to the admin after a reset. General registration listings omit both token columns. The one exception is the shared code used for individual sign-ups, which is public by design and can only add a sign-up.
- **Common attacks are blocked.** Registration and student fields are checked on the server, with text limits (for example 100 characters for a name and 500 for allergies). Error messages never reveal how the website works inside. Whatever is typed is handled safely, so it cannot be used to tamper with the database (SQL injection) or to run harmful scripts on other people's screens (cross-site scripting). The student consent tick-box is checked on the form, but the website does not keep a record that it was ticked.

- **The PDF export leaves out ages and allergies**, and browsers are told not to keep a copy of it. Once downloaded, though, the file is ordinary: store it carefully and delete it when done.
- **Browser protections.** The website tells browsers not to pass its addresses on to other sites, not to show it inside other sites, and to limit where scripts can load from. The admin dashboard loads no outside code at all.
- **It fails safe.** If the admin password or secret key is missing from the settings, the public pages keep working but the admin dashboard refuses *everyone*. If email fails, the registration is still saved.

Your responsibilities

- Keep the admin password to the smallest group that needs it, and change it if someone leaves (section 17).
- Keep the server settings private. They hold the password hash, the secret key and the email login.
- Tell teachers their **portal link is private**. If one is lost or shared, reset it from the dashboard and send the teacher the new one.
- Treat downloaded PDFs as confidential, and don't email them around.
- Make sure the hosting is backed up (section 7).

5 Going live on Railway, step by step

In one sentence: **move the GitHub repository to an SACTH account, deploy it on Railway, and pay for Railway's Pro plan.** This section explains why, and gives every step.

5.1 How it runs today

Item	Current state (last checked 5 October 2026)
Web address	`https://pou-hono-site.vercel.app`, a free sub-address of Vercel's own domain, which SACTH cannot own.
What works there	Only the pages display. **The server part isn't running**, so registering, student sign-up, the teacher portal, admin login and the home page counter all fail. No registration data is stored on Vercel.
Who owns it	Deployed from the GitHub repository `Melware0-0/PouHonoSite`. [confirm the Vercel account and plan with the team member who set it up]

Why not Vercel

Pou Hono keeps all its data in one database file beside the website. Vercel runs server code as short-lived "serverless" functions with no permanent disk, so registrations would fail to save or quietly disappear. Pou Hono needs a server that keeps running, with a permanent disk. Railway provides both.

5.2 Which Railway plan

Plan	Listed price	What matters for Pou Hono
Pro RECOMMENDED	$20 a month, which includes $20 of usage	**Allows sending email over SMTP**, which is how the confirmation email is built today. Volumes start at 50 GB (the database will be a tiny fraction of that).
Hobby	$5 a month, which includes $5 of usage	**Blocks outgoing SMTP email.** It would work only if a developer first switches the email to an HTTPS-based provider (section 6.2). Volumes up to 5 GB.
Free / Trial	$0	Not suitable for the live site: 500 MB volumes, SMTP blocked, and very small usage allowances.

Usage above the included amount is billed on top. Pou Hono is small (one server, one small file), so a modest bill is likely, but set a usage limit or alert in Railway's billing settings. Prices are as listed by Railway; confirm the currency and current figures at sign-up.

Sources, checked 12 October 2026: railway.com/pricing (plans, prices, volume sizes); docs.railway.com/networking/outbound-networking ("SMTP is only available on the Pro plan and above"; "Free, Trial, and Hobby plans must use transactional email services with HTTPS APIs"); docs.railway.com/reference/volumes (default volume sizes).

5.3 Where the data will live

Railway has four regions: US West (California), US East (Virginia), EU West (Amsterdam) and **Southeast Asia (Singapore)**. There is **no New Zealand or Australian region**, so the data would be held offshore, in Singapore as the nearest. This sits awkwardly with the data-sovereignty reason for building in-house. SACTH should make that call knowingly and say so in its privacy statement. If keeping data onshore is a must, use an NZ or Australian host with a long-running Node.js server and a permanent disk; the steps below still apply in spirit.

Source: docs.railway.com/reference/deployment-regions, checked 12 October 2026. Railway says more regions may be added, so check again before signing up.

5.4 The steps

Steps 1–3 are for SACTH. Steps 4–12 are best done by a developer with SACTH watching, so SACTH knows where everything is.

1. **Create a GitHub organisation for SACTH** (free), using a shared SACTH email with at least two SACTH owners. The current owner of `Melware0-0/PouHonoSite` then uses *Settings → Transfer ownership* on the repository to move it into that organisation. GitHub redirects the old address. Developers stay on as collaborators. Agree code ownership and licence in writing; the repository has no licence file.
2. **Create the Railway account with the same SACTH email**, not a developer's. Add a payment method, choose the **Pro** plan, and invite developers as members of the workspace.
3. **Decide the web address** (5.5), so it is ready for step 9.
4. **New Project → Deploy from GitHub repo**, and pick the repository in SACTH's organisation (Railway will ask to connect to GitHub). Railway builds it and starts it with `npm start`. It reads the Node.js version (22 or 24) from the project's settings file.
5. **Choose the region before adding storage.** In the service's settings, set the region (Singapore is nearest). Moving a service with storage between regions later causes downtime while the data is copied.
6. **Add a volume** (Railway's word for a permanent disk) to the service, mounted at `/data`. Run **one copy** of the service only. Railway does not allow extra copies (replicas) on a service with a volume, and the database is one file in any case.
7. **Set the variables** in the service's *Variables* tab:

Variable	Value
`NODE_ENV`	`production`
`ADMIN_PASSWORD_HASH`	The scrambled admin password. A developer generates it from a new, long password (section 17). Keep the password itself in SACTH's password manager.
`SESSION_SECRET`	A long random string, generated fresh (section 17).
`DATABASE_PATH`	`/data/pou-hono.db`
`TRUST_PROXY`	`1`, so the attempt limits see each visitor's real address. Check it after deploying (step 10).
`PUBLIC_BASE_URL`	The final address, e.g. `https://register.sacth.nz`. Every student link, portal link, QR code and email is built from it.
`EMAIL_ENABLED`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`	From the email provider (section 6). Leave `EMAIL_ENABLED=false` until the provider is set up.
`RAILPACK_DEPLOY_APT_PACKAGES`	`fonts-dejavu-core`, so the PDF export can print macrons and ‘okina (see the box below).
`PORT`	**Do not set.** Railway sets it.

8. **Switch off the demo data before the first real start.** On an empty database the website creates five example schools and eighteen example children. Before the first start on Railway, a developer should switch off the two "seed" blocks in `db.js` and push that change (section 17). This matters because the two example *individual* sign-ups cannot be removed through the website afterwards (section 11). If the site has already started with demo data, the cleanest fix is to stop the service, delete the database file and its `-wal` and `-shm` companions on the volume, and restart with the seed switched off, while nothing real is in it.

9. **Add the custom domain** in the service's *Networking* settings. Railway shows a `CNAME` record and a `TXT` record. Add *both* at the domain's DNS provider; the domain won't verify with the CNAME alone. Railway then issues the HTTPS certificate automatically. DNS changes can take a while to spread, up to 72 hours. Do this **before** any real teacher registers, so no one is ever sent a link on a temporary address.
10. **Test end to end** on the real domain:
    - Register a test class with your own email address. Check the email arrives (and isn't in spam), and that its links and QR code work.
    - Scan the QR code on a phone and sign up a test student with a macron in the name, e.g. "Tāne Ngāti".
    - Open the teacher portal from the email and check the student is there.
    - Sign in to the admin dashboard, check the figures, **export the PDF and check the macron prints correctly**.
    - From two different networks (e.g. office wifi and a phone on mobile data), use a throwaway deployment to exhaust a registration allowance on one network, then confirm the other still has its own allowance. Two successful registrations alone do not prove that client addresses are distinct.
    - Delete the test class (this also removes its test student).
11. **Turn on backups** (section 7).
12. **Retire the Vercel deployment** so it can't be mistaken for the real site.

> **The PDF export and special characters**
>
> The PDF needs a font with Māori and Pasifika characters (macrons such as ā, and the ʻokina). The website looks for one of a few common fonts already on the server. If none is there it falls back to a basic font and those characters print as junk. Railway's servers may not have one. Railway's build system documents the `RAILPACK_DEPLOY_APT_PACKAGES` variable for adding system packages, and `fonts-dejavu-core` is the standard package that provides the DejaVu font the website looks for. This has **not yet been tried on Railway**, so check it: after deploying, the server log must *not* contain "PDF export: no Unicode font found", and the macron test in step 10 must pass.
>
> Sources, checked 12 October 2026: railpack.com/guides/installing-packages; docs.railway.com/builds/build-configuration.

5.5 What SACTH needs to own

Asset	How SACTH owns it
Domain name	**Simplest:** a sub-address of a domain SACTH already controls, e.g. `register.sacth.nz`. Nothing to buy; whoever manages that domain's DNS adds Railway's two records. **Or buy a new name** (e.g. `.nz` or `.org.nz`) from an authorised .nz registrar, in **SACTH's or The Cause Collective's legal name**, never a developer's, with auto-renew on.
Source code	The SACTH GitHub organisation (step 1).
Hosting	The SACTH Railway account (step 2).
Email account	The email provider account (section 6), under the same SACTH email.
Passwords & keys	SACTH generates its own for production, stores them in a password manager, and enters them only in Railway. After handover, rotate them so no developer holds live credentials.
The data	The single database file on the Railway volume, and its backups. SACTH decides how long it is kept (section 11) and who can download it.

Every account above uses a shared SACTH email with at least two SACTH owners, so access never depends on one person.

6 Setting up the confirmation email

The email is built and tested. It stays switched off until SACTH has an email provider and has set up its domain.

6.1 What is sent, and to whom

- **Teachers only**, straight after they register a class. It contains a greeting, a "save the date" box with their event day and the venue address, the **student link and its QR code**, their **private portal link** (with a "keep this to yourself" warning), a line about event-day details, and SACTH's contact email and phone number. It is sent as both a formatted and a plain-text version. A preview is in Appendix A.
- **Students and individuals get no email.** Their forms do not ask for an email address, on purpose, to collect as little about children as possible.
- Email is sent in the background. If it fails, the teacher still sees the confirmation screen with both links, and the failure is written to the server log.
- While email is switched off, the website writes a copy of each email to the server log instead (leaving out the private portal link). That copy includes the teacher's name and email address, so the hosting logs hold personal details; keep access to them limited.

6.2 Choosing a provider

Any transactional email provider will do: for example Resend (Railway's own suggestion), SendGrid, Postmark or Mailgun, or SACTH's existing email system if it allows sending through "SMTP". Compare their current prices and free allowances yourselves; Pou Hono sends one email per class, so the volume is small. Open the account with the shared SACTH email.

Railway's Hobby plan blocks this kind of email

Pou Hono sends email over SMTP, the standard email-sending method. **Railway blocks SMTP on its Free, Trial and Hobby plans; it is available on Pro and above.** So there are two options:

1. **Use Railway Pro** (recommended in section 5) and any provider's SMTP settings. No code change.
2. **Stay on Hobby and use the provider's web API instead.** A developer replaces one small function, `sendEmail()` in `email.js`, with the provider's own code (an example for SendGrid is in the comment at the top of that file). Everything else about the email stays the same. Roughly an hour or two of developer time, plus testing.

Source: docs.railway.com/networking/outbound-networking, checked 12 October 2026. Railway also notes that after upgrading to Pro the service must be redeployed before SMTP works.

6.3 Setting up your domain so emails aren't marked as spam

Emails must come from an address on a domain SACTH controls, e.g. `noreply@sacth.nz`. Email systems check that the sender is allowed to send for that domain. The provider will show you a few DNS records to add, at the same place as the Railway records in section 5:

- **SPF**: a public list of which services may send email for your domain.
- **DKIM**: a digital signature on each email that proves it really came from you and wasn't changed on the way.
- **DMARC** (recommended): tells receiving systems what to do with emails that fail those checks.

Add the records exactly as the provider shows them, then press "verify" in the provider's dashboard. Like the Railway records, they can take a while to take effect. If SACTH's domain already sends email (for example through Microsoft 365 or Google Workspace), whoever manages it should add the provider to the *existing* SPF record rather than creating a second one.

6.4 The settings

Variable	What to put
`EMAIL_ENABLED`	`true` to send. Anything else (or not set) means log only.
`SMTP_HOST`	The provider's SMTP server name, from its dashboard.
`SMTP_PORT`	Usually `587` (the default). Use `465` if the provider says so.
`SMTP_USER`, `SMTP_PASS`	The SMTP username and password or API key the provider gives you. Treat the password like the admin password.
`MAIL_FROM`	The sender name and address, e.g. `SACTH NZ Tech Week <noreply@sacth.nz>`, on the domain you verified.

If `EMAIL_ENABLED=true` but `SMTP_HOST` or `MAIL_FROM` is missing, each email fails with a clear message in the log; it never pretends to have sent.

6.5 Testing it

1. Set the variables above and redeploy.
2. Register a test class using your own email address.
3. Check the email arrives, and look in the spam folder too. Open it on a phone and on a computer; check the QR code shows and both links open the right pages.
4. In Railway's logs, look for `[email] Confirmation sent to …`. If you see `[email] Could not send` instead, the line after it says why (usually a wrong password, port or unverified sender).
5. Delete the test class from the admin dashboard.

6.6 What to personalise before switching it on

A developer edits these in `email.js` (section 9 has the full list):

- **The event-day details line** (`detailsPlaceholder`). It currently says "Times, what to bring and arrival details will be sent closer to the day." Replace it with real times, what to bring and arrival or parking details once confirmed.
- **The contact email and phone number**, which are typed into the email in two places (the formatted and the plain-text versions).
- **The event name** (`EVENT_NAME`), and the wording of the subject line and greeting if wanted.
- The venue address comes from `EVENT_LOCATION` in `server.js`, and the date from `public/event-days.js`.

7 The database: is it big enough?

An honest assessment, because the team asked for this to be flagged.

7.1 What it is

Pou Hono stores everything in **SQLite**: one database file, read and written by the website itself, with no separate database server. It runs in "WAL" mode, which lets people read while someone else is saving. It is simple, cheap, and keeps the data in one file SACTH controls.

7.2 Will it cope with about 1,000 users?

> **Very likely yes**, on one server with a permanent volume. The workload consists of small database writes, but write times and peak traffic have not been measured.

But be clear about three things:

- **It has not been load tested.** Nobody has yet sent 1,000 people at the site at once. The likely pinch point is a school assembly where hundreds of students scan the QR code in the same minute. The current limit deliberately rejects submissions above 100 per class and internet address per hour (60 for individuals), with a 600/hour address ceiling. A large assembly sharing one network may hit these limits even if the database has spare capacity. Test both performance and the allowances before launch.
- **It limits you to one server.** You cannot run two copies side by side for extra capacity or to stay up if one fails; while the server restarts, the site is briefly down.
- **Backups are file copies.** There is no managed database service doing it for you; backups have to be set up (7.4).

7.3 When to move to a bigger database

Move to a database server such as PostgreSQL (Railway offers one) if any of these become true:

- You need **more than one copy** of the website, or it must stay up through a server failure (high availability).
- The **load test fails** and tuning doesn't fix it, or the project grows **well beyond** this event (many events, many thousands of sign-ups).
- You want **managed backups** and point-in-time restore from the provider.

What moving involves: it is a developer task of **days, not hours**. The database setup in `db.js` and every database query in `server.js` (35 `db.prepare()` call sites) would need rewriting for a Postgres connection, which also changes how the code waits for answers from the database. Then the data is copied across and everything retested. Doable, but not a settings change; for NZ Tech Week 2027 we don't recommend it unless the load test says otherwise.

7.4 What to do before launch

1. **Run a load test** on the Railway deployment before registrations open, simulating hundreds of students signing up within a few minutes. Run it with `EMAIL_ENABLED=false`, ideally on a separate copy of the site, and delete the test data afterwards. A short developer job.
2. **Turn on daily backups.** Railway offers volume backups on a schedule: daily (kept 6 days), weekly (kept 27 days) and monthly (kept 89 days), restored from the service's *Backups* tab. Check that your plan includes them. Two cautions from Railway's own documentation: backups can only be restored into the same project, and **wiping a volume deletes all its backups**. (docs.railway.com/reference/backups and /reference/volumes, checked 12 October 2026.)
3. **Also keep copies off Railway.** A developer regularly takes a safe copy with `sqlite3 /data/pou-hono.db ".backup backup-YYYY-MM-DD.db"` (never a plain file copy while the site runs) and stores it, encrypted,

somewhere SACTH controls.

8 Languages and translations

Visitors pick English, te reo Māori, gagana Sāmoa or lea faka-Tonga from the menu in the header. The choice is remembered on that device.

> **The translations are drafts and must be checked**
>
> All the Māori, Samoan and Tongan text was produced with AI and **has not been checked by fluent speakers**. It may be wrong, awkward or culturally off. It must be reviewed by someone fluent in each language before the site is used with real families. The warning at the top of `public/translations.js` says the same and should stay until the review is done.

8.1 What is translated

Page	Translated?
Header, menu and footer	YES on every page
Home page	YES
Register (teacher form, terms, confirmation)	YES
Student and individual sign-up	YES, except the heading on a class link ("Register for *school*..."), which is English
Teacher portal	YES
FAQ	ENGLISH ONLY (header and footer translate)
Admin dashboard, PDF export, confirmation email	ENGLISH ONLY

The five workshop names and their hosts stay in English in every language, because they are the hosts' official titles. The event dates also stay in English.

8.2 Getting them reviewed

1. Ask a developer to export the strings from `public/translations.js` into a spreadsheet: one row per string, with English beside each language. There are 117 strings per language.
2. Give each reviewer the English and their language, plus the screenshots in Appendix A so they see where each string appears. Record their names in section 13.
3. The developer puts the corrected text back into `translations.js`, checks each page in each language, and then updates the warning at the top of the file to say who reviewed it and when.

8.3 How it works (for the developer)

- Each piece of text on a page has a label, e.g. `<h1 data-i18n="reg_title">`. `translations.js` holds the text for each label in four blocks: `en`, `mi`, `sm` and `to`. Placeholders use `data-i18n-placeholder`; the few strings with formatting (such as the home page heading) use `data-i18n-html`.
- Messages that the page's own script shows (e.g. "Submitting...", "Copied!", a success message) use `i18nText('key', {name: …})` from the same file.
- **To change a string:** edit it in all four blocks. **To add one:** add the label to the HTML and the key to **all four** blocks. A key missing from a language shows the English text instead.
- **To add a fifth language** (e.g. Cook Islands Māori): add a new block to `translations.js` with every key, add a line for it to the language menu and to `LANG_LABELS` in `public/nav.js`, and get it reviewed like the others.

9 Personalising the website: what to edit where

For SACTH's web developer. Each change is a text edit, then a push to GitHub; Railway redeploys automatically. Afterwards, check the page in all four languages.

To change...	Edit
Event dates	`public/event-days.js` **only**. The form, portal, seed data, server checks and email date labels read it. The admin table formats each stored booking date independently. Keep the `YYYY-MM-DD` format. Classes already registered keep their stored date; fix those by hand. The current dates (18–20 May 2027) are **placeholders**.
Workshops (names, hosts, descriptions)	They must match everywhere: the `validSessions` list in `validateStudent` in `server.js`; the flip cards in `public/index.html` (name, host, icon and `aria-label`); the session cards in `public/student.html`; the session drop-downs in `public/teacher.html` and `public/register.html`; the descriptions `home_ws_*` in `public/translations.js` (all four languages); the "5 hands-on workshops" figure in `index.html` if the number changes; and the demo students in `db.js`. If real sign-ups exist, a rename must also update their stored value (see `RENAMED_SESSIONS` in `db.js`). The admin filter and chart follow automatically.
Event name ("NZ Tech Week 2027")	The value `register.html` sends (`session: 'NZ Tech Week 2027'`) and the matching check in `validateRegistration` in `server.js` must change together. Also: `EVENT_NAME` in `email.js`, the PDF title in `server.js`, page titles and text in every `public/*.html`, the student page heading in `student.html`, and the strings in `translations.js`.
Venue address and map	`EVENT_LOCATION` in `server.js` (used in the email); the map embed address and the address under "Find us" in `public/index.html`; the footer in `public/nav.js`; the FAQ answers in `public/faq.html`; and `home_about_p1` in `translations.js` (all four languages).
Contact email and phone	The footer in `public/nav.js` (email and phone, twice each: the link and the text), and the email in `email.js` (formatted and plain-text versions). The FAQ links to the footer and has no address of its own.
Email wording	`email.js`: `messageFor()` for the subject, introduction and workshop line; `buildConfirmationEmail()` for the greeting; `detailsPlaceholder` for event-day details; the header, contact line and footer inside `buildConfirmationEmail()`.
Logos	Replace `public/images/SACTH_Symbol-1.webp` and `public/images/tcc_logo_symbol.webp` (same names), or change the two `<img>` tags in `public/nav.js`.
Photos	Add files to `public/images/` and a matching `gallery__item` button in the gallery in `public/index.html`. Only use photos with consent from the people in them.
Prize banner ($500)	The banner and the "$500" figure in `public/index.html`, and `home_prize` and `home_stat_prize_label` in `translations.js`. Delete the banner block if there is no prize.
FAQ	`public/faq.html` (English only). Seven questions; copy an existing `faq-item` block to add one.
Terms and consent wording	Teacher terms: `disclaimer_text`; student statement: `student_disclaimer_text` and `student_checkbox_accept`, all in `translations.js` (four languages). Also update the English fallback text in `public/register.html` and `public/student.html`.
School list	`public/schools.js`, one name per line. It only suggests; any name can still be typed.
Colours	The colour settings at the top of `public/styles.css` (`--green`, `--bg` and so on). Some pages have extra styles of their own at the top of the page file.

Admin password	No file edit. Generate a new hash, replace `ADMIN_PASSWORD_HASH` in Railway, redeploy (section 17). Everyone is signed out.
Attempt limits	The `makeLimiter(...)` lines near the top of `server.js`.

10 Before going live: checklist

Nothing here needs new features built. These are decisions, content and setup.

	Action	Section	Who
☐	**GitHub repository moved to an SACTH organisation**	5.4	SACTH + developer
☐	**Railway account on the Pro plan, deployed with a volume**, all variables set	5.4	SACTH + developer
☐	**Demo data switched off** before the first real start	5.4, 17	Developer
☐	**Custom domain and HTTPS working**; `PUBLIC_BASE_URL` set to it	5.4	SACTH + developer
☐	**PDF export prints macrons** on Railway	5.4	Developer
☐	**Email provider set up**, domain verified, test email received	6	SACTH + developer
☐	**Event-day details and contact details** put into the email	6.6	SACTH + developer
☐	**Confirm the three event dates**	9	SACTH
☐	**Load test passed**; **daily backups** running, with off-site copies	7	Developer
☐	**Fluent-speaker review** of te reo Māori, gagana Sāmoa and lea faka-Tonga	8	SACTH
☐	**Real admin password** set, held by named people. The brief's example password, `SACTH2027`, is publicly known and must not be used.	17	SACTH
☐	**Review the wording** of the teacher terms, the student statement and the FAQ	9	SACTH
☐	**Confirm the $500 prize giveaway** banner, which is a public promise	9	SACTH
☐	**Privacy statement, parent/caregiver consent and retention plan**	11	SACTH
☐	**Fill in contacts and ownership**	13	SACTH
☐	**Full end-to-end test** on the real domain, then retire Vercel	5.4	SACTH + developer

11 Known limitations and recommended next steps

These are the edges of what was in scope, listed so nobody is surprised by them.

Privacy and consent PRIORITY

The website collects children's names, ages and health information, which puts it under the **NZ Privacy Act 2020**. The following are not yet in place:

- A **privacy statement** page explaining what is collected, why, who can see it, where it is held (Singapore, if Railway is used), how long it is kept and how to ask for it to be corrected or deleted. The student form has a one-paragraph statement, but there is no full policy.
- A **parent or caregiver consent** step. At the moment students agree for themselves, at any age.
- A **retention plan**: when the data and any downloaded PDFs and backups will be deleted after the event, and who does it.

We recommend SACTH settle these before registrations open. Building a privacy page and a consent tick-box is a small piece of work once the wording is agreed.

Removing a single student IMPORTANT

- The admin dashboard **cannot edit or remove an individual student**. (Version 1.1 of this document wrongly said it could.) Staff can only delete a whole class, which removes all its students.
- A student in a class can be removed by **their teacher**, from the teacher portal.
- An **individual sign-up cannot be removed by anyone through the website**: individuals have no teacher, and the shared "Individual / Walk-in" record cannot be deleted. This matters if someone asks for their details to be deleted (a privacy request), and it is why the two example individuals must be prevented by switching the demo data off before launch (section 5) rather than removed afterwards.
- Until an admin "remove student" feature is built (a small job), such requests need a developer to remove the row directly from the database.

Admin dashboard

- **One shared admin password**, with no individual staff accounts and no log of who changed what.
- **The PDF export is the only download.** There is no spreadsheet (CSV/Excel) export, the PDF ignores the dashboard filters, and there is no printable per-class list with allergies for the day. Teachers can see their own class's allergies in their portal.
- **The PDF font must be checked on the live server** (section 5), or names with macrons and ‘okina print incorrectly.
- **The dashboard loads every child's details.** To draw its figures it downloads all students, including names and allergies, even though the figures only need ages, year groups and workshops. Trimming this is recommended.
- Staff can't edit a class's year groups, and teachers can't change their own class details; the portal tells them to contact SACTH.
- On smaller laptop screens the registrations table scrolls sideways to reach the Status and Actions columns.

Email

- **Only teachers get an email.** Students and individuals get none, because no email address is collected.
- A teacher who loses their portal link can search their inbox for the confirmation email; otherwise an admin resets it and sends the new link by hand.

- On Railway's Hobby plan, email needs a small code change (section 6).

Other

- **The translations are unreviewed AI drafts** (section 8).
- Attempt limits are kept in memory, so they reset whenever the server restarts.
- There are **no automated tests** (changes are checked by hand), and no load test yet (section 7).
- The home page map loads from Google; it receives no registration data.
- The demo classes have no year groups recorded, because they were created before the question existed. (The screenshots in Appendix A show example year groups added for illustration.)

12 Not included in this project

These were either handled separately or deliberately left out.

Item	Status	Notes
Production hosting and deployment	TO DO	Fully prepared; follow section 5.
Email provider account and DNS	TO DO	The email itself is built; follow section 6.
Load testing to 1,000 users	RECOMMENDED	See section 7.
Uploading paper forms	NOT IN SCOPE	The database has space to record a file name, but no upload feature exists and no files are stored.
Admin per-student edit/remove, privacy page, consent step	NOT BUILT	Recommended next steps; see section 11.

13 Contacts and ownership

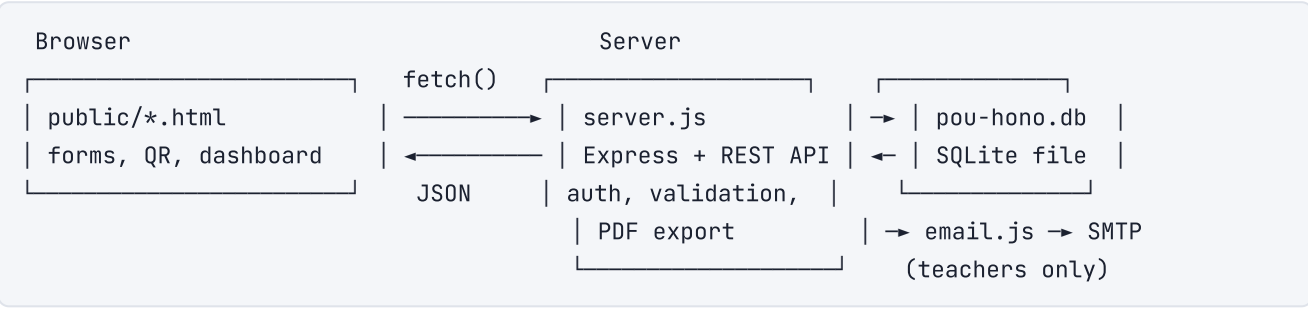
To be completed by SACTH and the developer at handover.

Role	Name and contact
Project owner (SACTH / The Cause Collective)	[name, email, phone]
Holder(s) of the admin password	[names]
Railway account owner	[account email, plan, billing contact]
Email provider account owner	[provider, account email, sending domain]
Domain name owner	[domain, registrar, account owner, who manages DNS]
GitHub organisation owner	[organisation, owners]
Who runs backups	[name, schedule, where copies are kept]
Technical support after handover	[name, email, support terms]
Translation reviewers	te reo Māori: [name] gagana Sāmoa: [name] lea faka-Tonga: [name]
Public contact email and phone on the website and in the email	`sacth@thecausecollective.org.nz` and `+64 9 869 2433`. Confirm these are correct.

PART B Technical reference for the developer who hosts, maintains or extends Pou Hono. The repository's `README.md` and `CLAUDE.md` cover the same ground and should be kept up to date alongside the code.

14 Architecture, file map and dependencies

Node.js + Express 5, a single-file SQLite database via `better-sqlite3`, and plain HTML/CSS/JavaScript served as static files. No framework, no bundler and no build step: you edit a file and reload the page.



File	Purpose
`server.js`	The entire back end: Express app, security headers, static serving of `public/`, rate limiters, the `requireAdmin` and `requireTeacherToken` middlewares, validation, every route (including the PDF export, drawn with PDFKit), `EVENT_LOCATION`, and the final error handler.
`db.js`	Opens the database at `DATABASE_PATH` or next to itself (WAL mode, foreign keys on), creates the schema including `admin_sessions`, adds missing columns, backfills tokens, renames the old placeholder sessions, creates the walk-in container, and seeds demo data on an empty database.
`email.js`	Builds and sends the teacher confirmation email with nodemailer over SMTP (or logs it while `EMAIL_ENABLED` is not `true`). `sendEmail()` is the only provider-specific function.
`public/index.html`	Home page: hero, live counter, prize banner, about, workshop flip cards, photo gallery with lightbox, map.
`public/register.html`	Teacher path (3 steps, year groups, school search, manual student entry) and the entry point for the individual path.
`public/teacher.html`	Teacher class portal, served at `/register/:token`.
`public/student.html`	Student sign-up, served at `/join/:token`, or directly for individuals.
`public/admin.html`	Admin dashboard (login gate, stats, filters, table with edit, reset teacher link and delete, PDF export button, Chart.js pie chart). English only.
`public/faq.html`	FAQ (English body).
`public/nav.js`	Injects the shared header (logos, menu, language picker) and footer (contact details) into every page.
`public/translations.js`	UI strings in `en`, `mi`, `sm`, `to`; `applyTranslations()` and `i18nText()`.
`public/event-days.js`	The three event dates. Loaded by both Node (`require`) and the browser (`<script>`), so they are defined once.
`public/schools.js`	South Auckland school names for the teacher form's `<datalist>` (suggestions only).
`public/styles.css`	Shared styles and colour variables.

`public/images/`, `public/vendor/`	Logos and eight event photos; self-hosted Chart.js 4.4.1 and its MIT licence.
`.env.example`	Committed template for the settings, with the commands to generate each value.

Dependencies

`package.json`: `express`, `better-sqlite3`, `bcryptjs`, `cookie-parser`, `express-rate-limit`, `qrcode`, `dotenv`, `nodemailer` (email) and `pdfkit` (PDF export). `engines.node` is `"22.x || 24.x"`. The only browser library is Chart.js, self-hosted. `npm audit` reported no vulnerabilities on 12 October 2026 (after the `proxy-addr` advisory fix in PR #18); recheck before launch.

npm 12 and the native SQLite module: npm 12 does not run package install scripts unless they are approved. `package.json` approves exactly `better-sqlite3@12.11.1` under `allowScripts`. If `better-sqlite3` is ever upgraded, update that entry too, or the native module won't be built and the server will fail on start. (If a local `npm ci` leaves `node_modules/better-sqlite3/build/Release/better_sqlite3.node` missing, run `npm rebuild better-sqlite3`.)

15 Database

One SQLite file, `pou-hono.db`, created on first run next to `db.js`, or wherever `DATABASE_PATH` points (on Railway, the volume). The default file is gitignored. SQLite also uses `-wal` and `-shm` companion files while running. Email, server logs, backups and downloaded PDFs can hold copies of some data; protect those too.

`registrations`: one row per class (plus the walk-in container)

Column	Type	Notes
`id`	INTEGER PK	Auto-increment.
`school`, `contact`, `email`	TEXT NOT NULL	Teacher's details. `school` is display text only, not an identity.
`students`, `adults`	INTEGER	Booked numbers (1–500 and 0–200).
`not_attending`	INTEGER, default 0	How many booked students won't come (0 to `students`). Head count = `students - not_attending`.
`year_groups`	TEXT, default `''`	Comma-separated, in school order, e.g. `Year 7, Year 8`. Required (at least one) on public registration; an admin edit that omits it keeps the stored value. Empty on older rows.
`session`	TEXT	Always `'NZ Tech Week 2027'` for public registrations. Workshops live on `students`.
`date`	TEXT `YYYY-MM-DD`	Event day, stored this way so it sorts and compares as text. Public registrations must use a day from `event-days.js`; admin edits may use any real date.
`notes`, `file_name`	TEXT	`file_name` is only a string. No file is stored.
`status`	TEXT, CHECK	`'Pending'` or `'Arrived'`.
`token`	TEXT UNIQUE	The **student sign-up** code (`crypto.randomUUID()`). The walk-in container's is public, via `/api/walk-in-registration`. Backfilled, with a unique index, on very old databases.

`teacher_token`	TEXT, unique index	The **teacher portal** credential. Returned only in `portalLink` at registration (and in the email) and by the admin reset route. NULL for the walk-in container.
`is_walk_in`	INTEGER 0/1	1 only for the shared container. A partial unique index guarantees at most one.
`created_at`	TEXT (UTC)	`YYYY-MM-DD HH:MM:SS`, shown in NZ time in the PDF. Set explicitly on registration; NULL ("—" in the PDF) on rows from before the column was added.

`students`: one row per signed-up student

Column	Type	Notes
`id`	INTEGER PK	
`registration_id`	INTEGER FK	`ON DELETE CASCADE`: deleting a class deletes its students.
`name`	TEXT NOT NULL	Up to 100 characters.
`age`	INTEGER, nullable	1–120 if given. Required on the student form, optional when a teacher adds a student.
`year_group`	TEXT NOT NULL	`Year 1` … `Year 13`, or `18+`.
`allergies`	TEXT	Free text, up to 500. Treat as sensitive health information.
`preferred_session`	TEXT NOT NULL	One of: `AI Fundamentals Workshop`, `Graphic Design Workshop`, `Build and Battle Robots Workshop`, `How to Hack a Bank (Ethical Hacking Workshop)`, `DJ'ing Basics Workshop`.
`created_at`	TEXT (UTC)	Set by SQLite.

`admin_sessions`: one row per admin sign-in

`id` (random 32-byte hex, also the signed cookie value), `created_at`, `expires_at` (8 hours on), and `password_fingerprint` (a hash of the configured password hash, so changing the password ends every session).

Upgrades, renames, the walk-in container and seed data

- **In-place upgrades:** `db.js` runs on every start. `not_attending`, `is_walk_in`, `teacher_token`, `token`, `created_at` and `year_groups` are each added through `addColumnIfMissing(...)` when absent, so older databases upgrade without being deleted.
- **Workshop rename:** on every start, students still holding an old placeholder value are moved to the real workshop: Programming → AI Fundamentals Workshop; Social Media → Graphic Design Workshop; Robotics → Build and Battle Robots Workshop; Computer Building → How to Hack a Bank (Ethical Hacking Workshop); Gaming → DJ'ing Basics Workshop. It is a no-op once done.
- **Walk-in container:** created on startup if missing (older databases have their existing "Individual / Walk-in" row marked). It cannot be edited (apart from status) or deleted through the API, so its students cannot be removed through the site.
- **Seed data:** on a *completely empty* registrations table, five example classes are inserted; the eighteen example students (two of them under the walk-in container) are inserted only on that same startup and only if the students table is empty. The seed does not modify existing real rows; the startup migrations described above can update existing rows.

16 Access control and API reference

Every route sits behind exactly one of three gates. When adding a route, decide its gate first.

- PUBLIC No credentials.
- TEACHER `requireTeacherToken` reads the **teacher token** from `Authorization: Bearer <token>` or `?token=`, looks it up by `teacher_token` only, loads that registration into `req.registration`, and scopes queries to it. A missing, unknown, student or walk-in token all return the **identical 404**.
- ADMIN `requireAdmin` checks the signed, httpOnly cookie `pou_hono_admin` (SameSite=Lax, `Secure` when `NODE_ENV=production`). Its value is a session id that must exist in `admin_sessions`, be unexpired, and match the current password fingerprint.

Method	Route	Gate	Purpose
POST	`/api/admin/login`	PUBLIC	Admin login (bcrypt compare). 10 attempts per 15 min per IP.
POST	`/api/admin/logout`	PUBLIC	Deletes the server session and clears the cookie.
GET	`/api/admin/session`	PUBLIC	Reports `{ authenticated }`.
GET	`/api/admin/students`	ADMIN	Every student in one query.
GET	`/api/admin/export/pdf`	ADMIN	A4 landscape PDF of all students (name, school or "Individual", year level, workshop, registered at) and teacher registrations (name, school, email, registered at), NZ time, page numbers. No ages or allergies. `Cache-Control: no-store`; file name `registrations-YYYY-MM-DD.pdf`.
GET	`/api/registrations`	ADMIN	All registrations (no token columns).
PUT	`/api/registrations/:id`	ADMIN	Full edit, or `{ status }` only for check-in. Walk-in: status only (409 otherwise).
POST	`/api/registrations/:id/teacher-link`	ADMIN	Issue a new teacher token (old portal link stops working); returns `{ portalLink }`. 409 for the walk-in container.
DELETE	`/api/registrations/:id`	ADMIN	Delete a class and, through the cascade, its students. 409 for the walk-in container.
GET	`/api/registrations/:id/students`	ADMIN	One class's students.
POST	`/api/registrations`	PUBLIC	Teacher registers a class (event days only, year groups required). Returns the record plus `link`, `portalLink` and `qrCode`, and fires the confirmation email without awaiting it. 20 per hour per IP.
GET	`/api/registrations/count`	PUBLIC	Home page counter (excludes walk-in).
GET	`/api/registrations/token/:token`	PUBLIC	School name and `isWalkIn` only, for the student page heading.
GET	`/api/walk-in-registration`	PUBLIC	The walk-in container's token, public by design.

POST	`/api/registrations/token/:token/students`	PUBLIC + token	Student sign-up with the student token. 600/hour per IP ceiling, then 100/hour per IP + class (60 for the walk-in token).
GET	`/api/my-registration`	TEACHER	The teacher's own record, including `year_groups`.
GET	`/api/my-registration/qr`	TEACHER	Student link (built from the student token) and QR code.
GET	`/api/my-registration/students`	TEACHER	Their students.
POST	`/api/my-registration/students`	TEACHER	Add a student by hand. 200 per hour per class.
DELETE	`/api/my-registration/students/:studentId`	TEACHER	Remove one of their own students only (scoped by `registration_id`).

There is no admin route to edit or delete an individual student, and none to delete a walk-in student.

Page routes

`/join/:token` (student token) serves `student.html`. `/register/:token` (teacher token) serves `teacher.html`. An unknown token on either, including a student token on `/register/`, redirects to `/`; a teacher credential somehow on the walk-in container gets a plain 404. Every other page is served statically from `public/`.

Cross-cutting behaviour

Every response carries `Referrer-Policy: no-referrer`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` and a Content-Security-Policy. `/api` responses are `Cache-Control: no-store`. Unknown `/api` paths return a JSON 404. A final error handler returns a generic JSON message (413 for oversized bodies) and never a stack trace. Share and portal links use `PUBLIC_BASE_URL` when set.

Rules that must not be broken

- **Never build SQL by string concatenation.** External values are bound using `db.prepare(...)` with `?` or named placeholders. Only trusted, code-defined column lists and migration identifiers are interpolated.
- **Select explicit columns** from `registrations` (see `REGISTRATION_COLUMNS`), never `SELECT *`, so a new column can't leak. `teacher_token` and `token` must never be added to it.
- The walk-in container is identified by `is_walk_in` via `isWalkInRegistration()`, **never** by its school name, and must never pass `requireTeacherToken`.
- In the front end, every user-supplied value inserted with `innerHTML` goes through the page's `escapeHtml` helper, or is set with `textContent`. The pages' `escapeHtml` does not escape quotes, so it must not be used inside HTML attributes. (`email.js` has its own, which does.)
- Email must never be awaited by a route, and must never include children's details.

17 Setup, configuration and operations

Run it locally

```
npm install                 # Node.js 22 or 24
cp .env.example .env        # Windows: copy .env.example .env, then fill in the values below
node server.js              # or: npm start   →  http://localhost:3000
```

Settings (`.env` locally, or Railway's variables)

Variable	Value
`ADMIN_PASSWORD_HASH`	bcrypt hash of the admin password. Generate with: `node -e "console.log(require('bcryptjs').hashSync(process.argv[1], 10))" 'NewPassword'`
`SESSION_SECRET`	Long random string that signs the admin cookie. Generate with: `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`
`PORT`	Optional; defaults to 3000. Leave unset on Railway.
`NODE_ENV`	`production` on the live server (HTTPS-only admin cookie).
`DATABASE_PATH`	Optional. Default: next to `db.js`. On Railway `/data/pou-hono.db`; the folder must exist.
`TRUST_PROXY`	Optional. A proxy hop count such as `1`, or an Express trust-proxy value, so rate limits see real visitor addresses. Trusted proxies must overwrite forwarding headers.
`PUBLIC_BASE_URL`	Optional. Canonical origin for share links, portal links, QR codes and emails, e.g. `https://register.sacth.nz`.
`EMAIL_ENABLED`	`true` sends; anything else logs instead (without the portal link).
`SMTP_HOST`, `SMTP_PORT`	The provider's SMTP server; port defaults to `587` (STARTTLS); `465` uses TLS from the start.
`SMTP_USER`, `SMTP_PASS`	SMTP credentials. If `SMTP_USER` is blank, no authentication is sent.
`MAIL_FROM`	Sender, e.g. `"SACTH NZ Tech Week <noreply@sacth.nz>"`, on a domain with SPF/DKIM set up.
`RAILPACK_DEPLOY_APT_PACKAGES`	Railway only, read by its build system, not by the app: `fonts-dejavu-core` for the PDF font (section 5).

If either admin secret is missing, the server starts with a warning and admin login refuses every attempt.

Routine tasks

- **Change the admin password:** generate a new hash, replace `ADMIN_PASSWORD_HASH`, restart. Every existing admin session ends.
- **Rotate the cookie secret:** replace `SESSION_SECRET` and restart. This also signs everyone out.
- **A teacher has lost or leaked their portal link:** use *Reset teacher link* on their class and send them the new link.
- **Disable demo data for launch (do this before the first production start):** in `db.js`, stop the class seed (the `if (rowCount === 0) { … }` block) and the student seed (the `if (seededRegistrationIds && studentCount === 0) { … }` block) from running, e.g. by changing each condition to `false`, then push. Deleting `pou-hono.db` alone does *not* give an empty site: on restart `db.js` sees an empty table and

seeds again. Deleting the five demo classes in the dashboard removes their students, but the two demo individuals under the walk-in container **cannot be removed through the site**; if they exist, either stop the server and delete the database file (with its `-wal` and `-shm` files) while it holds nothing real and restart with the seed disabled, or remove those two rows directly with SQL.

- **A privacy deletion request for one student:** a class student can be removed by the teacher in their portal. An individual (walk-in) student needs a developer to run, against the live database, `DELETE FROM students WHERE id = ?` with the right id (find it with a `SELECT` on name). Take a backup first.
- **Reset everything (development only):** stop the server and delete the database file with its `-wal` and `-shm` files. The next start rebuilds the schema and the demo data.
- **Back up:** don't copy the file while the server runs (WAL mode). Use `sqlite3 /data/pou-hono.db ".backup backup-YYYY-MM-DD.db"`, or Railway's volume backups (section 7). Keep copies off the server, encrypted.

Production checklist

- ☐ Railway Pro (or Hobby with an HTTPS email API in `sendEmail()`), a **volume** at `/data`, a single instance, region chosen before the volume.
- ☐ Seed blocks disabled in `db.js` before the first start; no demo rows in the live database.
- ☐ `NODE_ENV=production`, fresh `ADMIN_PASSWORD_HASH` and `SESSION_SECRET`, `DATABASE_PATH=/data/pou-hono.db`, `TRUST_PROXY` matched to the proxy, `PUBLIC_BASE_URL` set to the live HTTPS origin, `PORT` unset.
- ☐ HTTPS and the custom domain working. A QR code generated on the live site opens the live domain over `https://`.
- ☐ Rate limits see real visitor addresses: from two different networks, each gets its own allowance.
- ☐ Email: sender domain verified (SPF, DKIM, ideally DMARC); `EMAIL_ENABLED=true`, `SMTP_HOST`, `SMTP_PORT`, any required `SMTP_USER` / `SMTP_PASS`, and `MAIL_FROM` set; a test email received with working links and QR; `detailsPlaceholder` and contact details updated.
- ☐ No "no Unicode font found" warning in the log; a PDF export prints a macron name correctly.
- ☐ Build log shows Node 22 or 24 and `better-sqlite3` built; `npm audit` clean.
- ☐ Load test passed; daily backups running and one restore tried.

18 Making technical changes

Content and branding changes are in section 9. These are structural:

To change…	Edit
A new database column	Add it to `CREATE TABLE` in `db.js` *and* add an `addColumnIfMissing(...)` call. If it is user-facing, update `validateRegistration` / `cleanRegistration`, `REGISTRATION_COLUMNS`, `/api/my-registration`'s field list and the form. Internal columns must never be accepted from public request bodies.
A new API route	`server.js`. Copy a neighbouring route and choose its gate deliberately.
An admin "remove student" feature	A new `DELETE /api/admin/students/:id` behind `requireAdmin`, plus a button in `admin.html`. This is the recommended next feature (section 11).

A different email provider (HTTPS API)	Replace the body of `sendEmail()` in `email.js`; keep the QR as an inline attachment with content id `studentqr`. Add the provider's package to `package.json`.
A new outside resource (script, font, embed)	Add its origin to the Content-Security-Policy in the security-headers middleware near the top of `server.js`, or the browser will block it.
A new page	A new file in `public/`, copying an existing page's structure (nav placeholder, `styles.css`, `translations.js` then `nav.js`).
Upgrading `better-sqlite3`	Update the version in `allowScripts` in `package.json` to match (section 14).

19 Repository status

Everything is merged
`main` at `70fda57` (12 October 2026) contains all the work. No branches are awaiting merge. Merged branches can be deleted from GitHub.

PR	Branch	What it did
#2	`fix/separate-teacher-portal-token`	Separate student and teacher codes; the student link can't open the portal.
#3	`fix/teacher-link-rotation`	Admin can reset a teacher's portal link.
#4	`fix/rate-limits-and-proxy`	Limits that work for whole schools on one address; `TRUST_PROXY`, `PUBLIC_BASE_URL`.
#5	`fix/configurable-database-path`	`DATABASE_PATH`, for a host's permanent volume.
#6	`fix/admin-sessions`	Server-side admin sessions that expire and end on logout or password change.
#7	`fix/input-validation-and-errors`	Length and type limits; generic error messages.
#8	`fix/security-headers`	Browser security headers and a content security policy.
#9	`fix/self-host-chartjs`	Chart.js served from the site.
#10	`fix/adult-individual-age`	Age limit 25 → 120 for adult individuals.
#11	`fix/remove-default-admin-password`	No published password in the settings template.
#12	`fix/dependency-audit`	ip-address and qs updates.
#13	`fix/node-version-and-package-metadata`	Node version and package metadata corrected.
#14	`fix/home-counter-label`	Home counter says "classes".
#15	`fix/remove-ds-store`	Removed a stray macOS file.
#16	`docs/handover-v1.1`	Handover document v1.1.
—	`schoolsearch` (merge `01d2d22`)	School-name search (`public/schools.js`) and the `token`-column upgrade for very old databases.
—	`feature/workshops-i18n-pdf-email` (merge `9f99db2`)	Real workshops, home page and wider translations, PDF export, email foundation, year groups.
—	`email` (merge `7ef77f2`)	One confirmation-email system over SMTP with links, QR and save-the-date.
#18	`fix/railway-build-and-audit`	npm 12 `allowScripts` approval for `better-sqlite3`, `engines.node` "22.x || 24.x", `proxy-addr` advisory fix.

The teammate's three Railway prerequisites are all in place: `PORT` is read from the environment (`server.js`), the database location from `DATABASE_PATH` (`db.js`), and `package.json` has an `npm start` script.

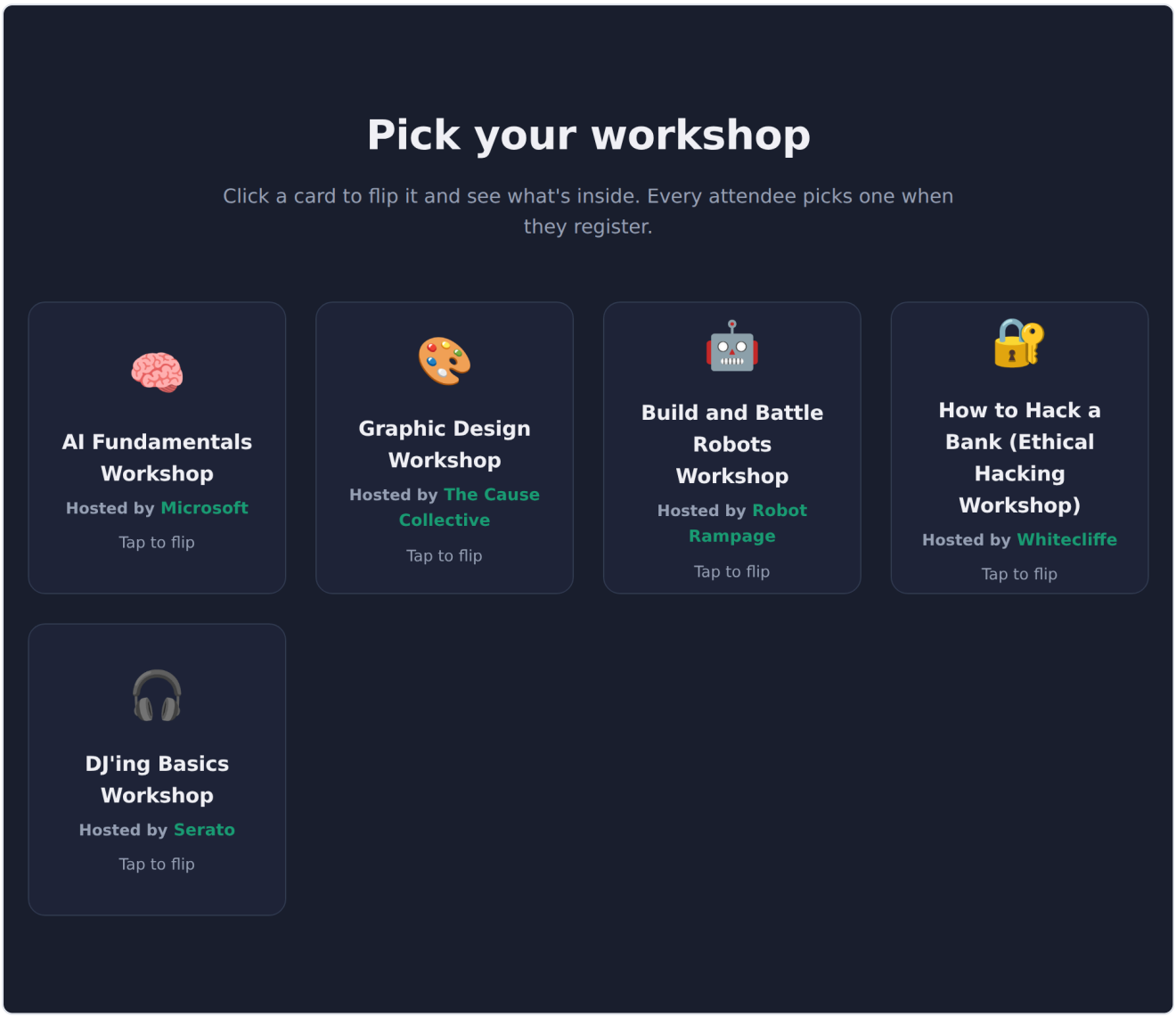
Documentation: `README.md`, `CLAUDE.md` (rules for developers and AI assistants) and `.env.example`; keep them current. No tests, linter or build step, by design. There is no project licence file (`package.json` says ISC); agree ownership and licensing in writing.

A Screenshots

Taken on 12 October 2026 from `main` running with its demo data (example year groups added to the demo classes).



Home page.

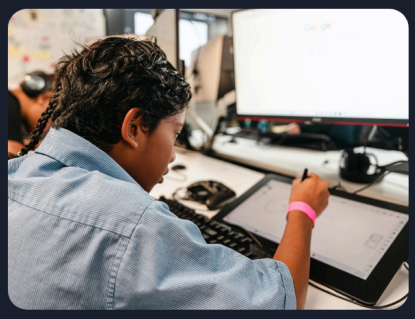


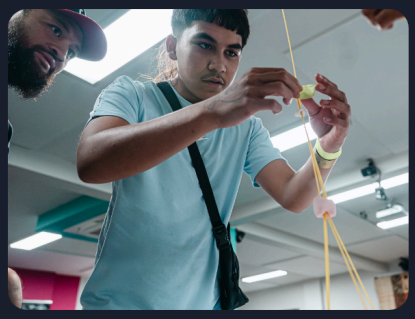
The five workshops. Each card flips to show a short description.

From past events

A few moments from previous SACTH events.





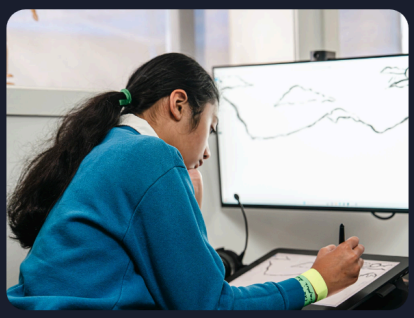
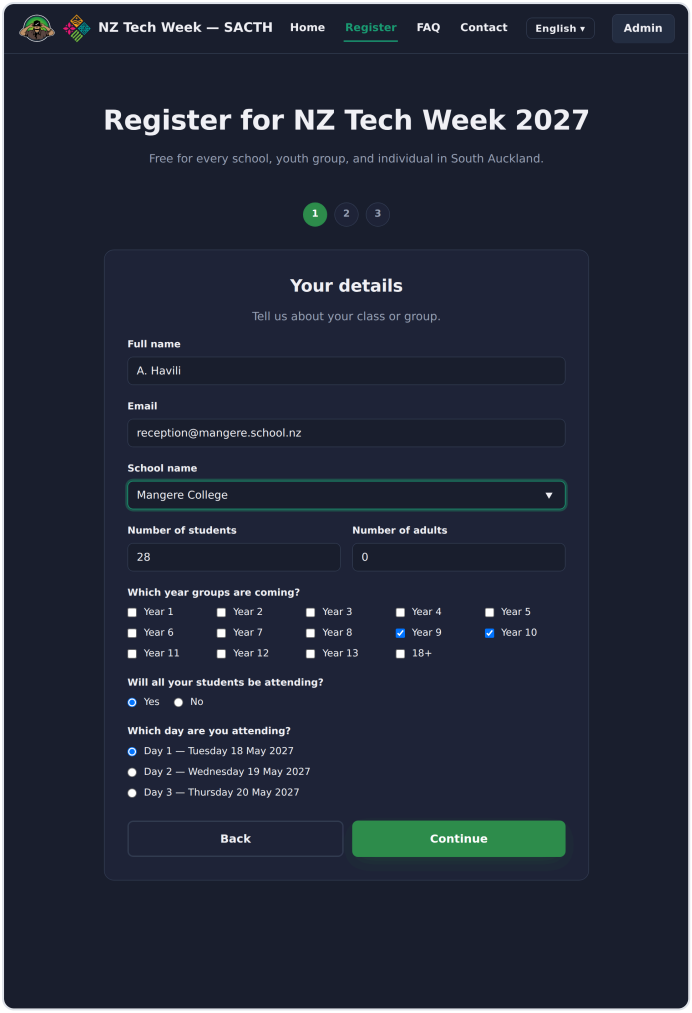


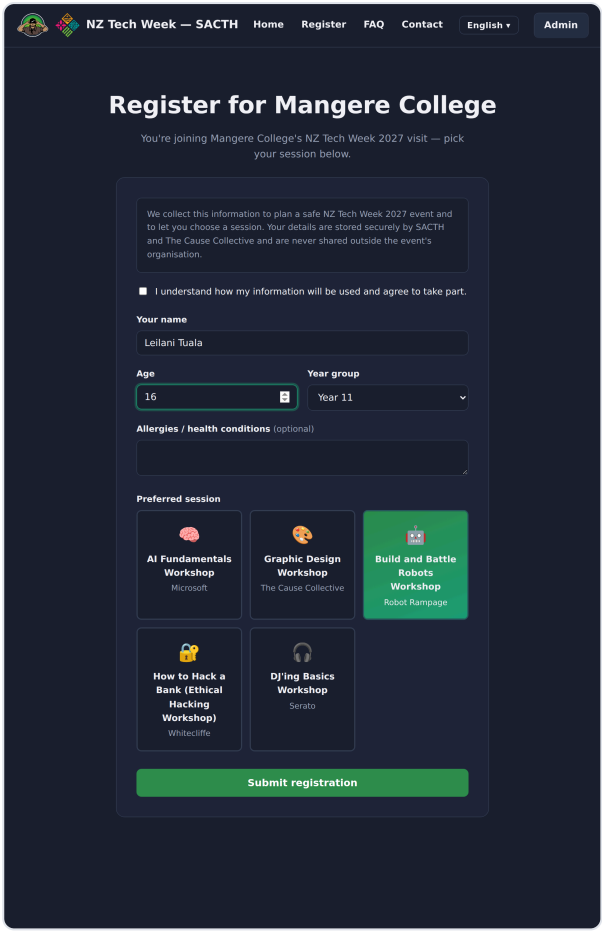
Photo gallery from past events. Each photo opens full-size.



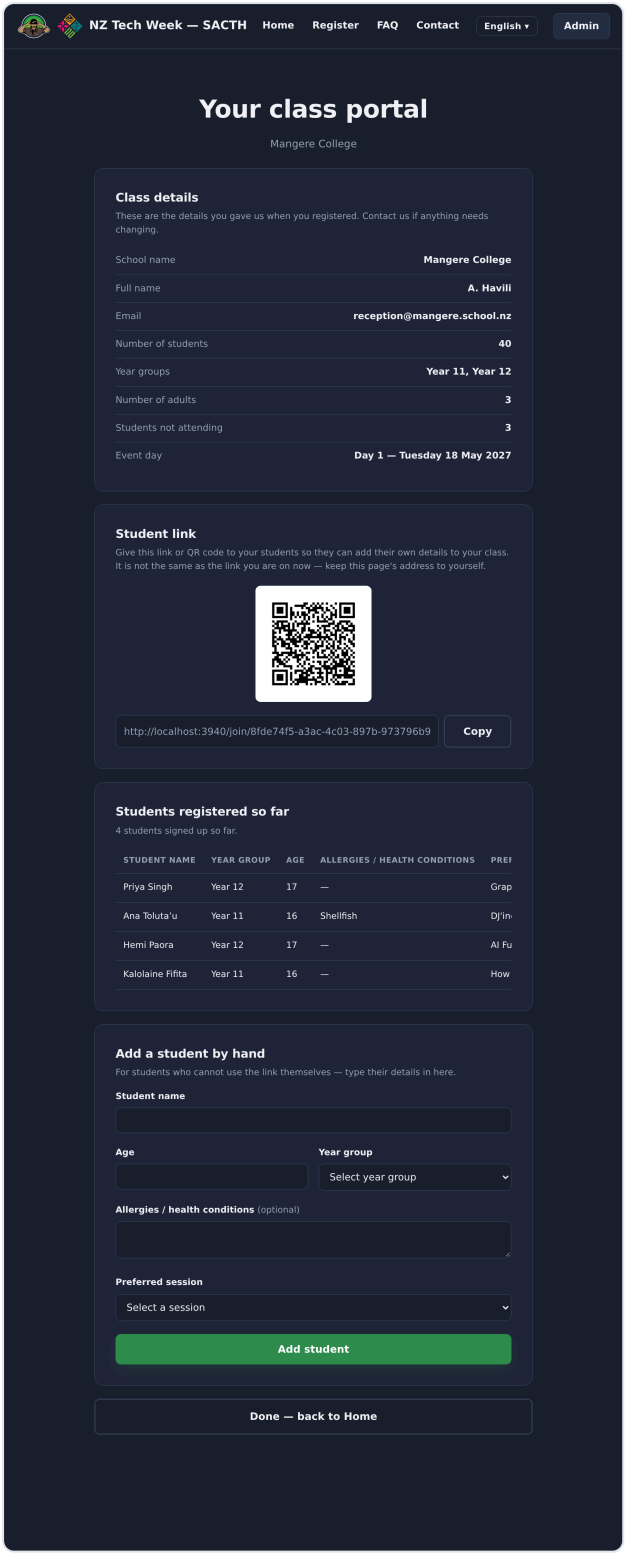
Register: choose teacher or individual.



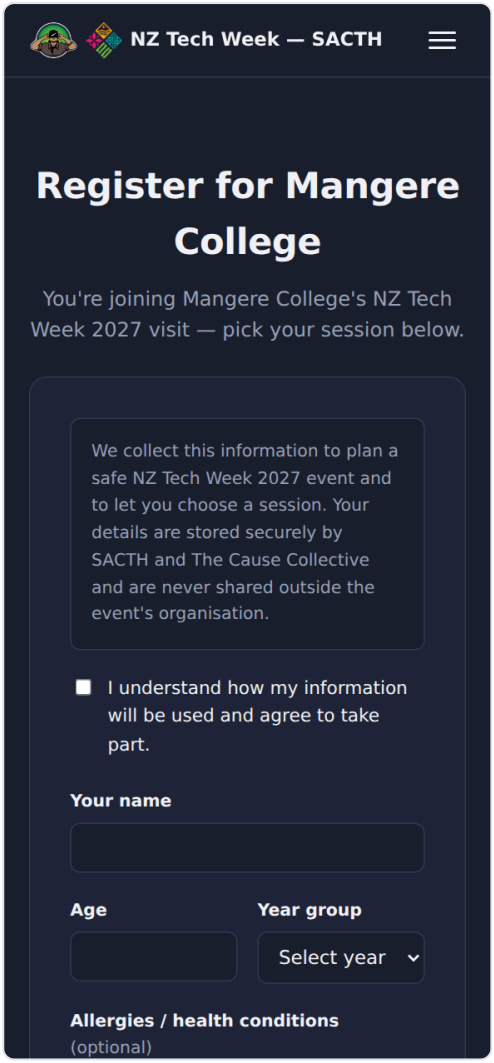
Teacher registration, step 1: school name with suggestions, year groups and event day.



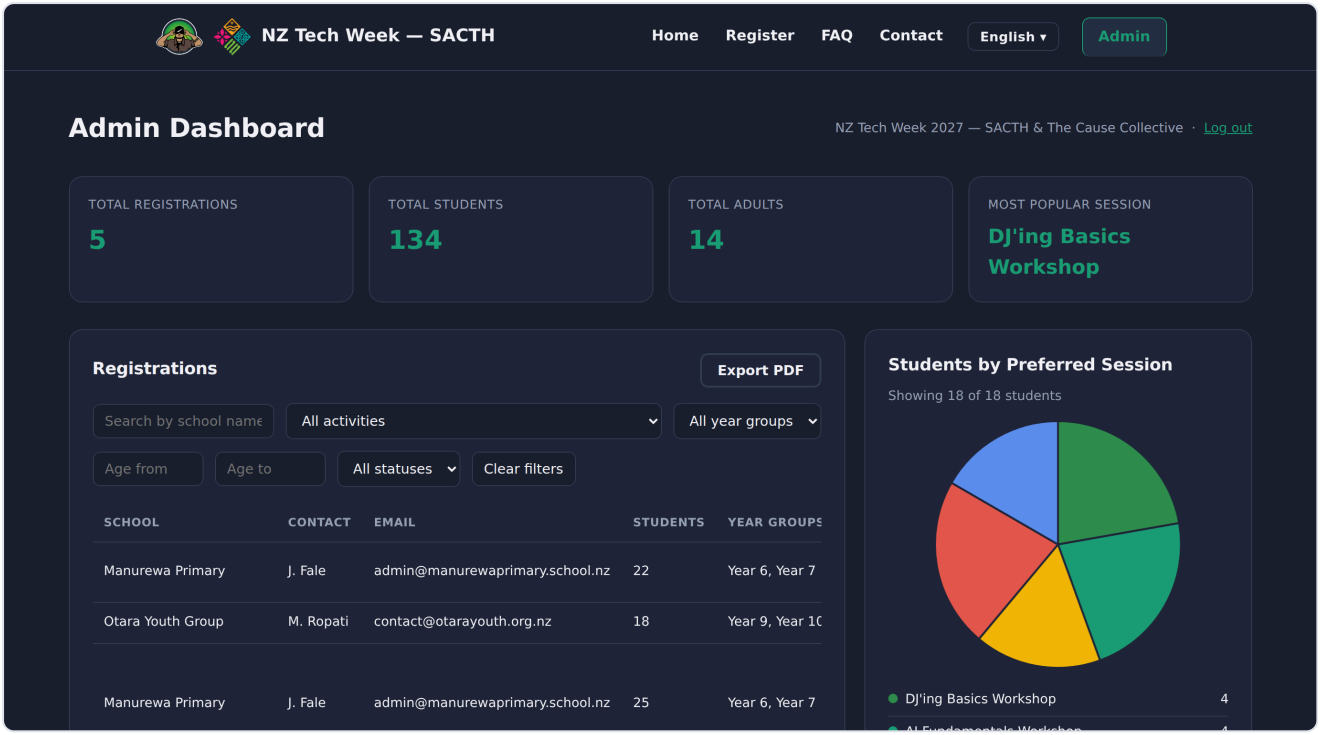
Student sign-up through a teacher's link.



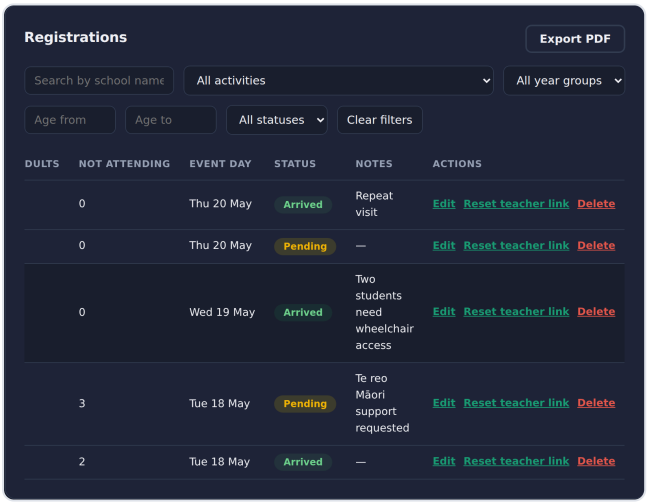
Teacher class portal: class details, student link and QR code, students so far, and adding a student by hand. The student table scrolls sideways to the Remove button.



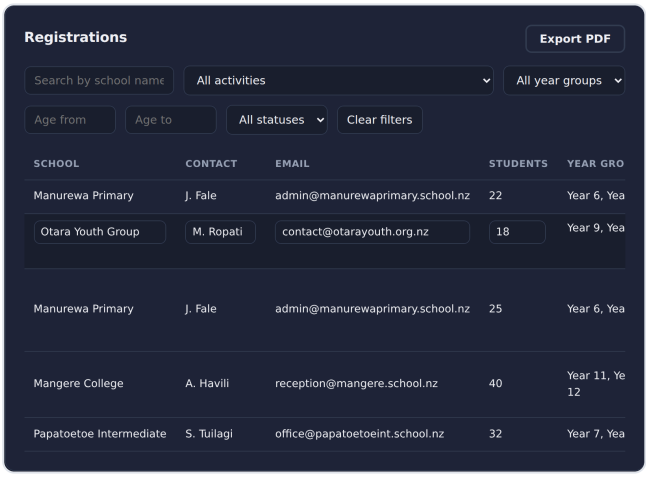
Student sign-up on a phone.



Admin dashboard, with the Export PDF button.



Registrations table: status (click to check in), Edit, Reset teacher link and Delete.



Editing a class in place. Year groups are shown but not editable.

NZ Tech Week 2027 — SACTH Registrations

Exported 12 Oct 2026, 7:15 pm (NZ time)

Student Registrations (18)

Name	School	Year Level	Workshop Selected	Registered At
Ana Toluta'u	Mangere College	Year 11	DJ'ing Basics Workshop	12 Oct 2026, 7:10 pm
Hemi Paora	Mangere College	Year 12	AI Fundamentals Workshop	12 Oct 2026, 7:10 pm
Kalolaine Fifita	Mangere College	Year 11	How to Hack a Bank (Ethical Hacking Workshop)	12 Oct 2026, 7:10 pm
Priya Singh	Mangere College	Year 12	Graphic Design Workshop	12 Oct 2026, 7:10 pm
Jacob Tuipulotu	Manurewa Primary	Year 6	How to Hack a Bank (Ethical Hacking Workshop)	12 Oct 2026, 7:10 pm
Lupe Faleolo	Manurewa Primary	Year 6	Graphic Design Workshop	12 Oct 2026, 7:10 pm
Nikau Heke	Manurewa Primary	Year 7	Build and Battle Robots Workshop	12 Oct 2026, 7:10 pm
Talia Ropati	Manurewa Primary	Year 6	How to Hack a Bank (Ethical Hacking Workshop)	12 Oct 2026, 7:10 pm
Tane Wiremu	Manurewa Primary	Year 6	DJ'ing Basics Workshop	12 Oct 2026, 7:10 pm
Anaru Rewiti	Otara Youth Group	Year 10	AI Fundamentals Workshop	12 Oct 2026, 7:10 pm
Deshaun Kaur	Otara Youth Group	Year 10	Build and Battle Robots Workshop	12 Oct 2026, 7:10 pm
Sina Tialavea	Otara Youth Group	Year 9	Graphic Design Workshop	12 Oct 2026, 7:10 pm
Aroha Ngata	Papatoetoe Intermediate	Year 7	Build and Battle Robots Workshop	12 Oct 2026, 7:10 pm
Mele Latu	Papatoetoe Intermediate	Year 8	AI Fundamentals Workshop	12 Oct 2026, 7:10 pm
Riya Patel	Papatoetoe Intermediate	Year 7	Build and Battle Robots Workshop	12 Oct 2026, 7:10 pm
Sione Vaka	Papatoetoe Intermediate	Year 8	DJ'ing Basics Workshop	12 Oct 2026, 7:10 pm
Josiah Leota	Individual	Year 9	AI Fundamentals Workshop	12 Oct 2026, 7:10 pm
Maia Thompson	Individual	Year 9	DJ'ing Basics Workshop	12 Oct 2026, 7:10 pm

The PDF export, first page (students; teacher registrations follow). No ages or allergies.

SACTH & THE CAUSE COLLECTIVE

NZ Tech Week 2027

Kia ora A. Havili,

Thank you for registering Mangere College for NZ Tech Week 2027. We can't wait to see you all there.

Your students choose their own workshop when they sign up using your class link.

SAVE THE DATE

Day 1 — Tuesday 18 May 2027

15 Earl Richardson Avenue, Wiri, Auckland 2104

Student sign-up link

Share this link, or the QR code, with your students so they can add themselves to your class.

https://register.example.nz/join/8fde74f5-a3ac-4c03-897b-973796b9e93b

Your private teacher link

Keep this to yourself — anyone with it can manage your class. Use it to see who has signed up and add students by hand.

https://register.example.nz/register/••••••••-example

Times, what to bring and arrival details will be sent closer to the day.

Questions? Email sacth@thecausecollective.org.nz or call +64 9 869 2433.

You're getting this email because you registered for NZ Tech Week 2027. A Cause Collective & SACTH initiative.

The teacher confirmation email (example; the portal link is masked here). The line "Times, what to bring..." is the placeholder to replace.

B Event-day check-in guide for staff

One page for whoever is on the door. Print it out for the day.

Before the day

1. Make sure you can sign in: go to [live website address] `/admin.html` and enter the admin password.
2. Check the **Notes** column for anything you need to prepare, such as wheelchair access or language support.
3. If you want a paper list of names, use **Export PDF**. It has no allergies; ask teachers to bring their class list from their portal, which shows each student's allergies and health notes.

When a class arrives

1. Type the school name into **Search by school name**.
2. If the school has more than one booking, check the **Event day** column to find today's.
3. Click the yellow **Pending** label. It turns green and reads **Arrived**. Click it again to undo a mistake.

Useful views

- **Who hasn't arrived yet?** Set the status filter to *Pending*.
- **A class that never registered?** Register them on the spot through the normal Register page, then mark them Arrived.
- **Numbers changed at the door?** Click **Edit** on the class, correct the student or adult numbers, and save.
- **Clear filters** resets the view.

Remember

- Sign-in lasts up to 8 hours. If the dashboard asks for the password again, sign in again; nothing is lost.
- **Delete** removes a class *and all its students* permanently. You don't need it on the day.
- Printed or downloaded lists contain children's names: collect them at the end of the day.
- Log out on shared computers when you're done.